

UNIVERSITY OF TRANSPORT HO CHI MINH CITY



INFORMATION SECURITY

SECURITY VULNERABILITIES

PhD. Tran The Vinh

GENERAL KNOWLEDGE OF SECURITY VULNERABILITIES



What is a computer security vulnerability?

A computer system vulnerability is a flaw or weakness in a system or network that can be exploited to cause damage or allow an attacker to manipulate the system in some way.



Types of security vulnerabilities:

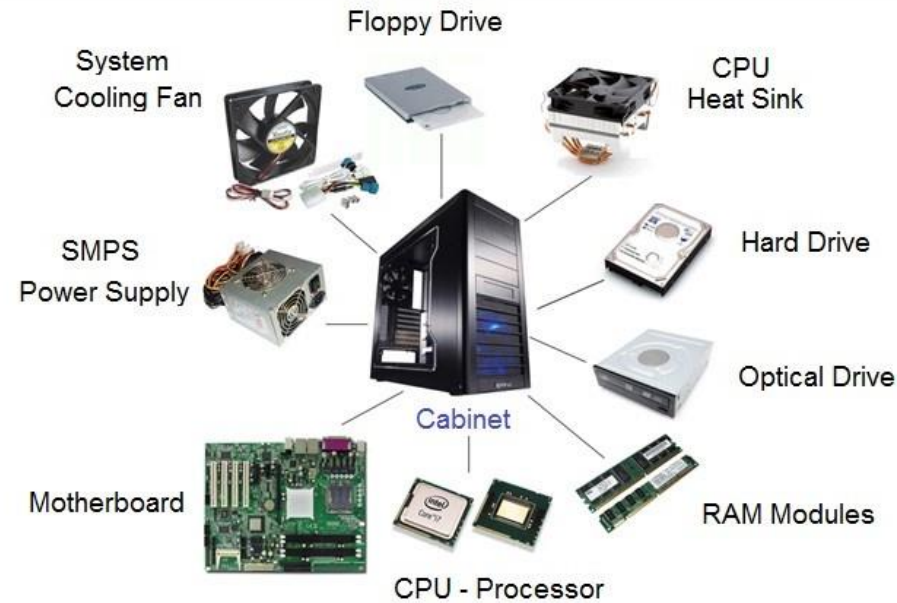
- **Operating System (OS) Vulnerabilities:** These are vulnerabilities within a specific operating system that hackers can exploit to gain access to the content on which the OS is installed.
- **Network Vulnerabilities:** These are issues occurring with network hardware or software that make the network susceptible to unauthorized external intrusion. Examples: insecure Wi-Fi access points, poorly configured firewalls.
- **Process Vulnerabilities:** Certain vulnerabilities can be created by specific process control measures. Example: the use of weak passwords.
- **User Vulnerabilities:** The weakest link in many cybersecurity architectures is the human factor. User error can easily expose sensitive data, create exploitable access points for attackers, or disrupt systems.

GENERAL KNOWLEDGE OF SECURITY VULNERABILITIES

COMPONENTS OF A COMPUTER SYSTEM

Hardware System

Computer System - Internal Hardware Components

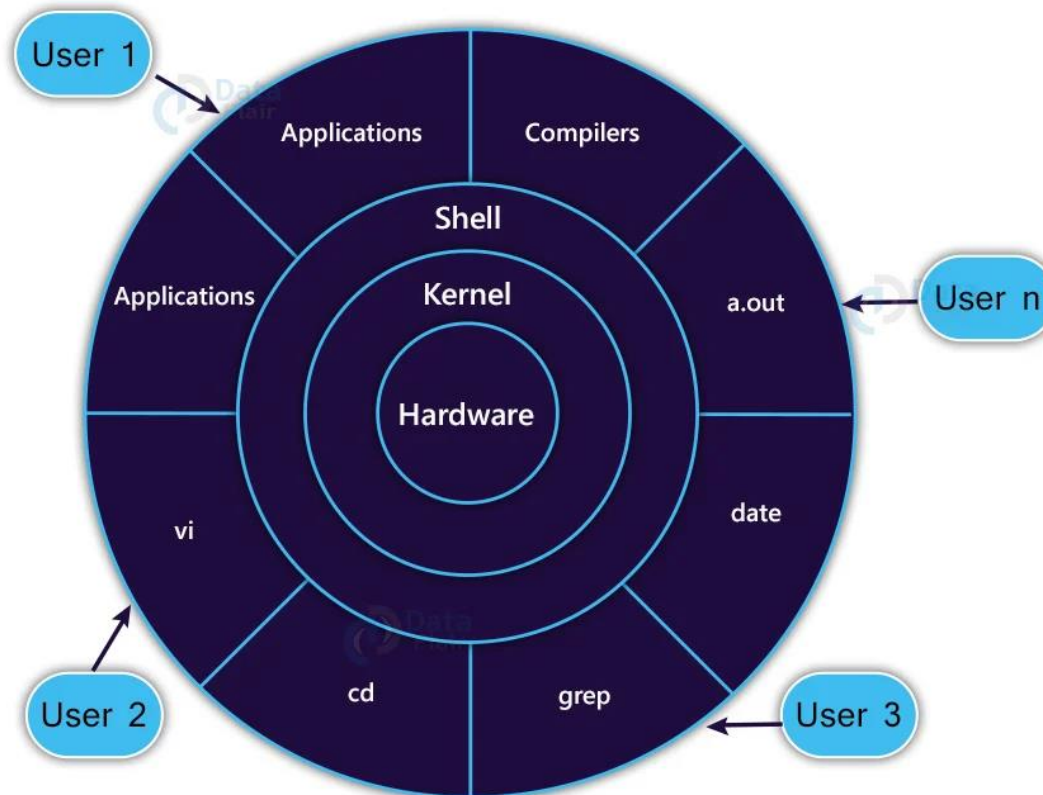


Software System

- Operating System (OS)
 - OS Kernel, device drivers
 - Service providers, utilities, etc.
- Application Software
 - Services (Web servers, Database/DB, DNS, etc.)
 - Web browsers, communication applications, etc.
 - Office suites, programming applications.

UNIX/LINUX OPERATING SYSTEM MODEL

Architecture of OS Linux



GENERAL KNOWLEDGE OF SECURITY VULNERABILITIES



COMPUTER SYSTEM WEAKNESSES:

Definition: These are errors or flaws (in design, installation, hardware, or software) that exist within the system.

- DDoS attack
- Pharming
- Effects:
 - Data destruction.
 - Data manipulation.
 - Data modification.
 - Data theft.
 - Identity theft.
- Vulnerabilities:
 - Physical vulnerabilities
 - Weak passwords
 - Insecure models
 - Insecure software
 - Natural disasters



SECURITY VULNERABILITIES

Definition: A security vulnerability is a weakness existing within a system that allows an attacker to exploit and compromise:

- Confidentiality.
- Integrity.
- Availability.

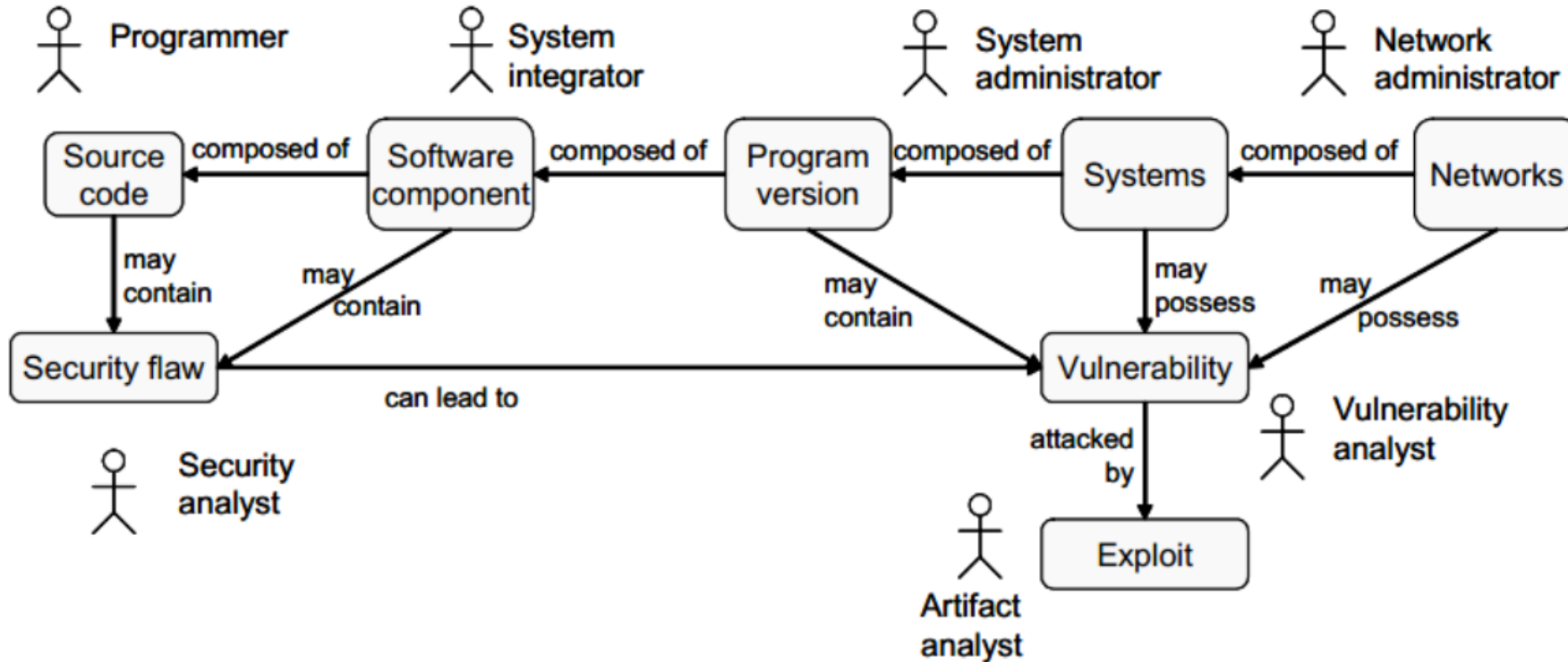
Depending on their **exploitability**, security vulnerabilities vary in their levels of **severity**.



GENERAL KNOWLEDGE OF SECURITY VULNERABILITIES



RELATIONSHIP MODEL BETWEEN OBJECTS AND ROLES IN THE SYSTEM



- Programmer.
- System integrator.
- System administrator.
- Network administrator.
- Security analyst
- Vulnerability analyst
- Artifact analyst.

- Source code
- Software component
- Program version
- Systems
- Networks
- Security flaw
- Vulnerability
- Exploit

GENERAL KNOWLEDGE OF SECURITY VULNERABILITIES



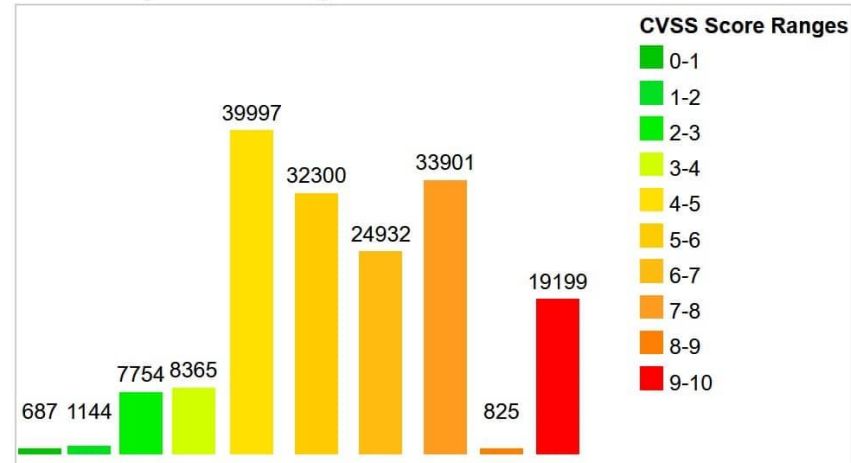
Current CVSS Score Distribution For All Vulnerabilities

Distribution of all vulnerabilities by CVSS Scores

CVSS Score	Number Of Vulnerabilities	Percentage
0-1	687	0.40
1-2	1144	0.70
2-3	7754	4.60
3-4	8365	4.90
4-5	39997	23.70
5-6	32300	19.10
6-7	24932	14.70
7-8	33901	20.00
8-9	825	0.50
9-10	19199	11.40
Total	169104	

Weighted Average CVSS Score: **6.5**

Vulnerability Distribution By CVSS Scores



SYSTEM VULNERABILITIES

Common types of security vulnerabilities in operating systems and application software:

- Memory safety violations
- Input validation errors
- Access-control problems
- Weaknesses in authentication, authorization, or cryptographic practices.
- Other security vulnerabilities.

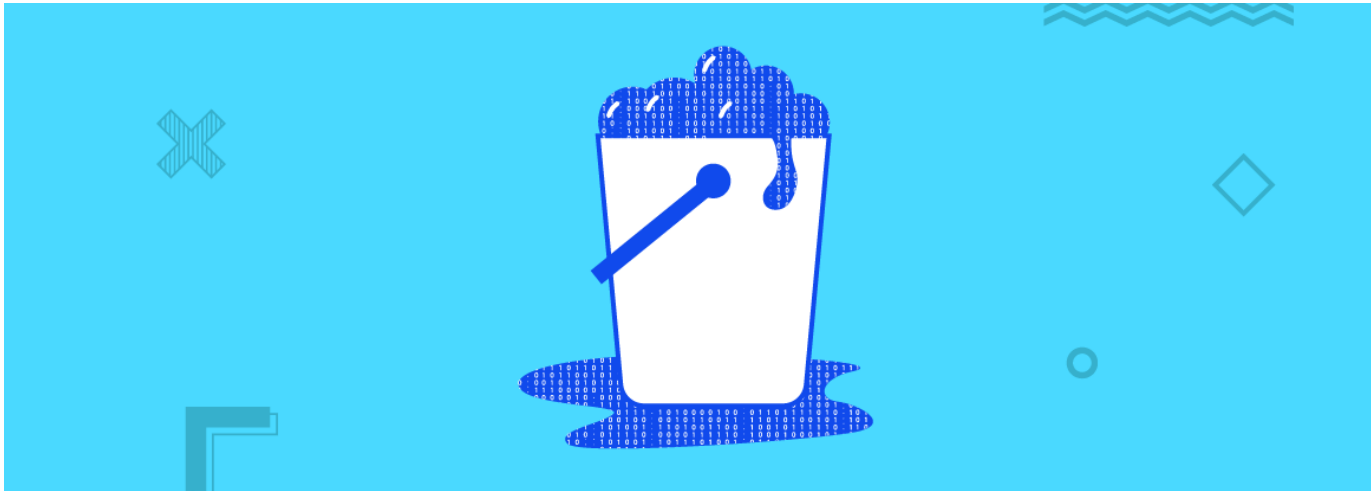


SYSTEM VULNERABILITIES

MEMORY SAFETY VIOLATIONS

Buffer overflow:

A buffer overflow occurs when an application attempts to write data beyond the boundaries of a buffer (either the end or the beginning of the buffer).



- Buffer overflows can cause applications to crash, lead to data loss, or even allow an attacker to take control of the system.
- Buffer overflows are software programming errors and account for a large percentage of security vulnerabilities.
- Not all buffer overflows are exploitable by an attacker.

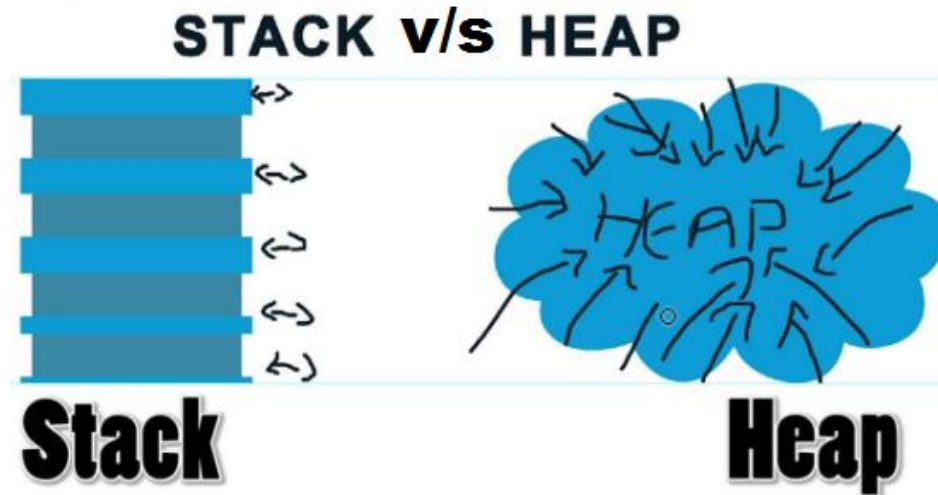
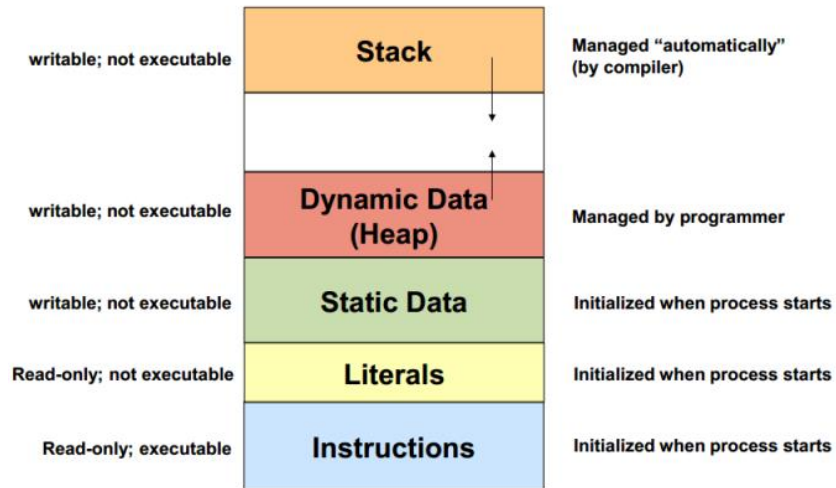
SYSTEM VULNERABILITIES

MEMORY SAFETY VIOLATIONS



Application buffer memory areas:

Stack: A linear data structure that operates on a **LIFO (Last In, First Out)** mechanism. This means the element added last to the stack will be the first one to be removed.



Heap: The memory area on RAM used for **dynamic allocation** tasks. This memory area is limited; when too many dynamic allocation tasks are performed and the allocated memory is not released (**memory leak**), a **buffer overflow** (or heap overflow) can occur.

SYSTEM VULNERABILITIES

MEMORY SAFETY VIOLATIONS



Buffer overflow prevention measures:

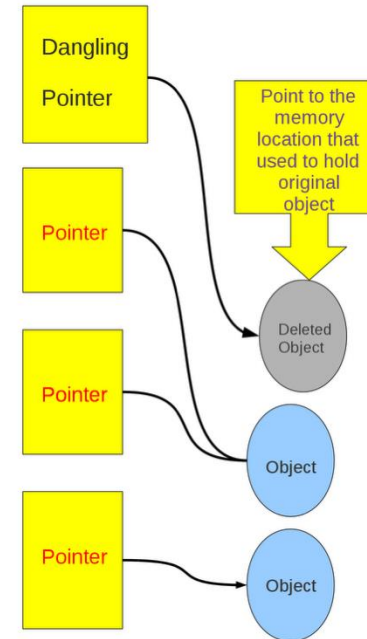
- **Choice of programming language:** Many languages provide **runtime checks** and issue warnings.
- **Use of secure libraries:** Implement secure libraries such as **The Better String Library**, **Safe C String Library (SafeSTR)**, or **Vstr**.
- **Executable Space Protection:** This technique prevents code execution in the stack, commonly known as **DEP (Data Execution Prevention)**.
- **Use of Stack Protection Mechanisms:**
 - Adding a random value (**canary**) before the **return address**.
 - Verifying this random value before returning to the calling program to determine if the return address has been modified.

SYSTEM VULNERABILITIES

MEMORY SAFETY VIOLATIONS

DANGLING POINTERS:

- A **dangling pointer** is a pointer that does not point to a valid object of the appropriate type. This is a specific case of a **memory safety violation**.
- Dangling pointers occur when an object is deleted or deallocated without modifying the value of the pointer to **null**, causing the pointer to still reference the memory location where the data was previously stored. Since the system may **reallocate** the previously freed memory (including to other processes), a corrupted pointer can lead to **unpredictable program behavior**. When a program writes data to memory using such a pointer, silent **data corruption** may occur, resulting in errors that are extremely difficult to detect.
- This type of vulnerability is very dangerous and, along with **memory leaks**, occurs very frequently.



Prevention Measures for Dangling Pointers:

- Some programming languages significantly reduce the likelihood of dangling pointers (e.g., Java, Python, Go, JavaScript, Rust, etc.).
- Methods to improve **memory access security**.

SYSTEM VULNERABILITIES

IMPROPER INPUT VALIDATION

- Input data must be validated to ensure it meets format and size requirements.
- Typical types of input data that require validation:
 - Text fields
 - Commands passed via URLs to trigger programs
 - Audio, image, or graphic files provided by users or other processes
 - Command-line arguments
 - Data from networks or untrusted sources

Hackers can test input data and try all possible permutations to exploit vulnerabilities.



SYSTEM VULNERABILITIES

IMPROPER INPUT VALIDATION



SQL INJECTION INTO TEXT FIELDS

Dim searchString, sqlString

' Case 1: Normal Input

searchString = "iPhone 14"

sqlString = "SELECT * FROM tbl_products WHERE product_name = " & searchString & ""

' ==> Generated Query: SELECT * FROM tbl_products WHERE product_name = 'iPhone 14'

' Case 2: Malicious Input (SQL Injection)

searchString = "iPhone 14'; DELETE FROM tbl_products; --"

sqlString = "SELECT * FROM tbl_products WHERE product_name = " & searchString & ""

' ==> Generated Query: SELECT * FROM tbl_products WHERE product_name = 'iPhone 14'; DELETE FROM tbl_products; --'

SYSTEM VULNERABILITIES

IMPROPER INPUT VALIDATION



SQL INJECTION FOR AUTHENTICATION BYPASS

ASP code + SQL server

Dim username, password, sqlString

' Case 1: Normal Login Attempt

username = "Antoanthongtin"

password = "lohonghethong"

sqlString = "SELECT * FROM tbl_users WHERE username = " & username & " AND password = " & password & ""

' ==> Generated Query: SELECT * FROM tbl_users WHERE username = 'Antoanthongtin' AND password = 'lohonghethong'

' Case 2: Authentication Bypass via SQL Injection

username = "aaaa' OR 1=1 --"

password = "aaaa"

sqlString = "SELECT * FROM tbl_users WHERE username = " & username & " AND password = " & password & ""

' ==> Generated Query: SELECT * FROM tbl_users WHERE username = 'aaaa' OR 1=1 --' AND password = 'aaaa'

SYSTEM VULNERABILITIES

IMPROPER INPUT VALIDATION



PREVENTION MEASURES:

- Validate all input data, especially data provided by users and from untrusted sources.
- Check the size and format of input data.
- Verify the integrity and sanity of data content.
- Create filters to sanitize special characters and language keywords where necessary, which could be utilized by attackers:
 - Special characters: *, ', =, --
 - Keywords: SELECT, INSERT, UPDATE, DELETE, DROP, ...

SYSTEM VULNERABILITIES

ACCESS CONTROL ISSUES



Access control involves controlling who (**subjects**) can access what (**objects**)?

Access control can be established by the operating system or by each individual application, and typically consists of two steps:

- **Authentication:** Verifying the identity information of the subject;
- **Authorization:** Granting access rights to the subject after their identity has been verified.

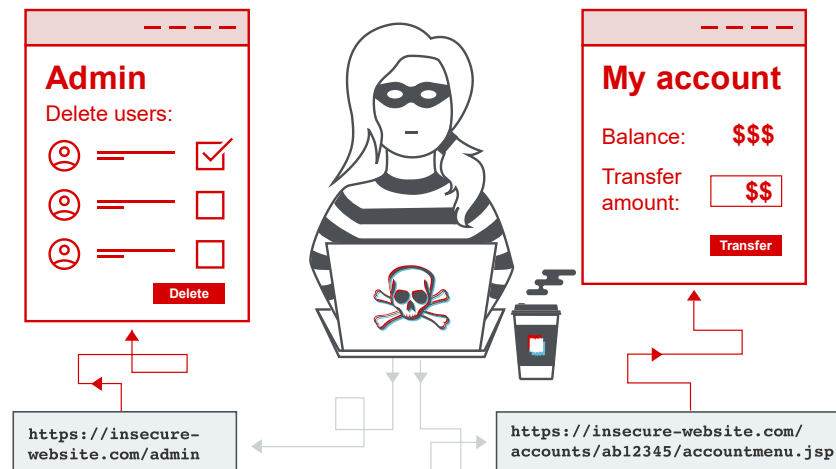
Subjects are granted access rights to the system at different levels based on the organization's **security policy**.

If access control fails, a regular user could **escalate privileges** to an administrator and gain full access to the system.

An attacker can exploit **security vulnerabilities** in the access control system to access files within the system.

An application running with **administrative privileges** has full access to the system:

- If an attacker gains control of the program, they will have full access to the system.



SYSTEM VULNERABILITIES

ACCESS CONTROL ISSUES



PREVENTING ACCESS CONTROL VULNERABILITIES

Access control vulnerabilities can generally be prevented by implementing a **defense-in-depth** approach and applying the following principles:

- **Never rely solely on obfuscation** for access control.
- Unless a resource is intended for public access, **deny access by default**.
- Whenever possible, use a **centralized access control mechanism** across the entire application to enforce policies.
- At the code level, mandate that developers **explicitly declare allowed access** for each resource and deny access by default.
- Thoroughly **audit and test** access controls to ensure they function as designed.

SYSTEM VULNERABILITIES

ISSUES WITH AUTHENTICATION, AUTHORIZATION, AND CRYPTOGRAPHY



Authentication:

- Passwords are stored in **plain text** → high risk of password exposure during the transmission of authentication credentials.
- Using simple, predictable passwords, or using the same password for an extended period.
- Implementing weak authentication mechanisms: for example, certain authentication mechanisms of the **HTTP protocol**.

Authorization:

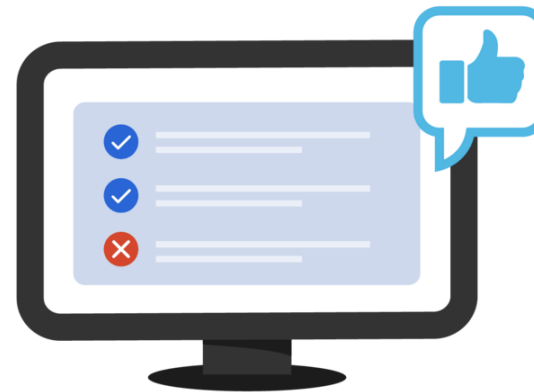
- Weak authorization enforcement mechanisms that are easily **bypassed**.

Authentication



Confirms users
are who they say they are.

Authorization



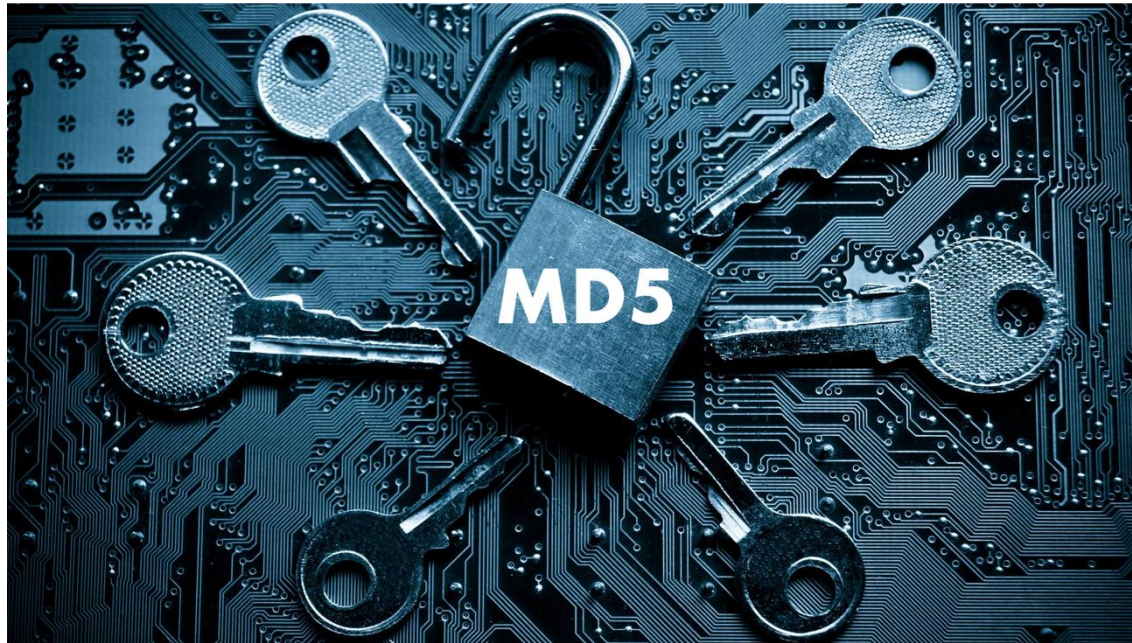
Gives users permission
to access a resource.

SYSTEM VULNERABILITIES

ISSUES WITH AUTHENTICATION, AUTHORIZATION, AND CRYPTOGRAPHY

ISSUES WITH CRYPTOGRAPHIC SYSTEMS

- **Using weak, obsolete, or vulnerable encryption/decryption algorithms and hash functions** (e.g., DES, MD4, MD5, etc.).
- **Using weak encryption/decryption keys:**
 - Short key length.
 - Predictable keys.
- **Secret key exchange issues.**
- **Sender/Recipient authentication issues.**
- **High computational overhead** (especially for public-key/asymmetric encryption systems).



SYSTEM VULNERABILITIES



Insecure file operations:

- Performing read/write operations on files stored in locations where other users also have write permissions.
- Failing to properly validate the file type, device identifier, links, or other file attributes before use.
- Failing to check return codes/return values after each file operation.
- Assuming a file with a local path is a local file and bypassing verification procedures:
 - Remote files can be mapped into the local file system → obtaining a local path.

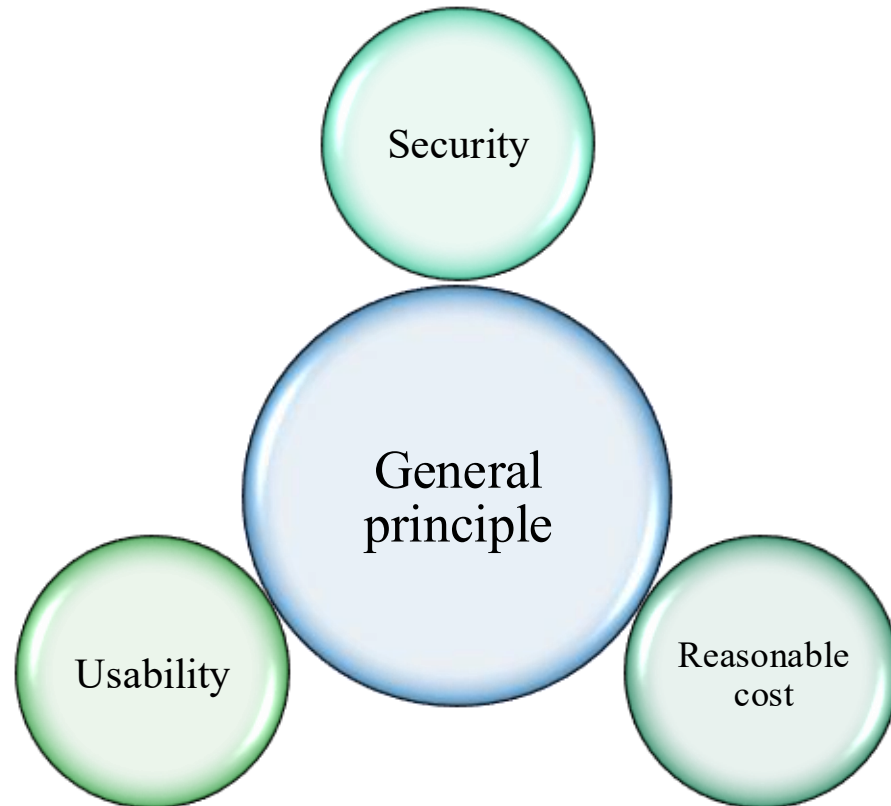
Race conditions:

- A race condition exists when a change in the sequence or timing of two or more events causes a change in the system's behavior.
- This is a type of flaw/error where the program can only function correctly if events occur in a specific, intended order.
- An attacker can exploit the time window between two events to inject malicious code, rename files, or interfere with the normal operation of the system.

VULNERABILITY MANAGEMENT AND REMEDIATION

General Principles

- Managing and remediating security vulnerabilities, as well as enhancing system resilience, must be carried out according to the general principle of balancing **Security**, **Reasonable Cost**, and **Usability**.
- The specific meaning of this principle is to ensure system security at an appropriate level with a reasonable cost, while the system must remain usable—meaning it retains the capability to provide full and useful features to users.



VULNERABILITY MANAGEMENT AND REMEDIATION

SPECIFIC MEASURES

- Regularly monitor and update information regarding security weaknesses and vulnerabilities from official websites::
 - <http://cve.mitre.org/> (CVE - Common Vulnerabilities and Exposures)
 - <http://www.cvedetails.com/> (CVE Details)
 - <http://web.nvd.nist.gov/> (National Vulnerability Database)
 - <https://owasp.org/www-community/vulnerabilities/>
- **Periodically update patches** and upgrade the operating system and application software.
- **Use patch management systems** and automated periodic updates:
 - Microsoft Windows Updates.
 - Update utilities on Linux.
 - Auto-update features of applications (e.g., Google Update service).
- For **critical vulnerabilities**, immediate updates (out-of-band updates) are required.

VULNERABILITY MANAGEMENT AND REMEDIATION

SPECIFIC MEASURES

- Strict user management, password, and access control policies must be established at both the operating system and application levels:
 - Users should only be granted the **minimum necessary permissions** (least privilege) to perform their assigned tasks.
 - Granting permissions beyond what is required can lead to **privilege abuse**.
- Implement perimeter defense measures such as Firewalls and Proxies:
 - Block services and ports that are not strictly necessary.
 - Maintain **logs** of all network access activities.
- Utilize vulnerability scanners and anti-malware software:
 - These can mitigate the risks of being exploited or having security vulnerabilities leveraged by attackers.